

Git, GitHub & Bash Scripting

ArIES IITR — Robotics & AI Development

Sections covered

A	The Linux Terminal & Bash Commands
B	Bash Scripting
C	Git — Version Control
D	GitHub — Remote Collaboration
E	Advanced Git & GitHub Actions
F	Assignment

A

The Linux Terminal & Bash Commands

Note

Reference: The Missing Semester of Your CS Education — <https://missing.csail.mit.edu/2020/shell-tools/>

Reference: GNU Bash Manual — <https://www.gnu.org/software/bash/manual/>

What is the terminal?

The terminal (also called the shell or command line) is a text interface to your computer. Instead of clicking icons, you type commands. On Linux and macOS the default shell is **Bash** (Bourne Again SHell). Almost every robotics computer — Jetson, Raspberry Pi, any Linux SBC — is controlled primarily through the terminal.

Tip

Ubuntu/Linux → Ctrl + Alt + T

macOS → Cmd + Space, type Terminal, press Enter

Windows (WSL) → open Ubuntu from the Start menu

The prompt

When you open a terminal you see a **prompt** — a line waiting for input. Everything after the \$ is what you type. Press **Enter** to run.

```
prompt
username@hostname:~/Documents$
# username = you, hostname = machine name, ~/Documents = current folder
# $ means the shell is ready
```

Navigating the file system

Linux organises files in a directory tree. Your home folder is ~ (short for /home/yourname). The root of the whole tree is /.

```
navigation
pwd                # print current directory
ls                 # list files
ls -l              # long listing – permissions, size, date
ls -la             # include hidden files (names starting with .)
ls -lh             # human-readable sizes (KB, MB)
cd Documents       # move into 'Documents'
cd ..              # go up one level
cd ~               # go to home directory
cd /               # go to root
cd -               # go back to previous directory
cd ~/projects/ros_ws # move to an absolute path
```

Tip

Tab = auto-complete a command or path. Up/Down arrows = scroll command history.

Creating and managing files

files and directories

```
mkdir my_folder          # create a directory
mkdir -p a/b/c           # create nested directories in one step
touch hello.py           # create an empty file
cp hello.py backup.py    # copy a file
cp -r folder/ new_folder/ # copy an entire directory
mv hello.py src/hello.py # move or rename a file
rm hello.py              # delete a file (permanent – no undo)
rm -r old_folder/        # delete a directory and everything inside
rmdir empty_dir          # delete an empty directory
```

Warning

rm is permanent. There is no Recycle Bin. Double-check before running rm -r.

Reading file contents

reading files

```
cat file.txt             # print entire file
less file.txt            # page through a file (Q to quit)
head file.txt            # first 10 lines
head -n 20 file.txt      # first 20 lines
tail file.txt            # last 10 lines
tail -f robot.log        # follow a file live – great for logs
wc -l file.txt           # count lines
```

Searching

search

```
grep 'ERROR' robot.log   # lines containing 'ERROR'
grep -i 'error' robot.log # case-insensitive
grep -r 'import rclpy' src/ # search all files in a directory
grep -n 'def main' node.py # show line numbers
find . -name '*.py'       # find all Python files
find . -name '*.bag' -size +100M # bag files larger than 100 MB
```

System information

```

system info
whoami                # current username
hostname              # machine name
uname -a              # OS and kernel info
df -h                 # disk usage
free -h               # RAM usage
top                   # live process monitor (Q to quit)
ps aux                # list all running processes
kill 1234              # stop process with PID 1234
kill -9 1234          # force-stop a process

```

File permissions

Every file has permissions for three groups: **owner**, **group**, **others**. Permissions are **r** (read), **w** (write), **x** (execute).

```

permissions
ls -l script.sh
# -rwxr-xr-- 1 ubuntu ubuntu 234 Jun 1 script.sh
# rwx = owner can read, write, execute
#   r-x = group can read, execute
#       r-- = others can only read

chmod +x script.sh      # give execute permission to all
chmod 755 script.sh     # rwxr-xr-x
chmod 644 config.yaml   # rw-r--r--
sudo command             # run as administrator

```

Pipelines and redirection

| sends the output of one command as input to the next. > writes output to a file. >> appends to a file.

```

pipelines
ls -la | grep '.py'      # list files, keep only .py lines
ps aux | grep ros2       # running processes, filter for ros2
cat log.txt | wc -l      # count lines in a file

echo 'Hello' > hello.txt  # write to file (overwrites)
echo 'World' >> hello.txt  # append to file
ls > filelist.txt         # save listing to a file

command &                # run in background
Ctrl+C                   # stop a running command
Ctrl+L                   # clear screen
history                   # show recent commands

```

Package management

```

apt
sudo apt update           # refresh package list
sudo apt install git       # install git
sudo apt install git curl nano # install multiple packages
sudo apt remove nano       # uninstall
apt search ros2            # search for packages

```

Quick reference

Command	What it does	Example
pwd	Current directory	pwd
ls	List files	ls -lh
cd	Change directory	cd ~/robot_ws
mkdir	Create folder	mkdir src
touch	Create empty file	touch node.py
cp	Copy file or folder	cp a.py b.py
mv	Move or rename	mv old.py new.py
rm	Delete (permanent)	rm log.txt
cat	Print file	cat config.yaml
grep	Search text	grep 'error' log.txt
find	Find files	find . -name '*.py'
chmod +x	Make executable	chmod +x run.sh
sudo	Run as admin	sudo apt install git
echo	Print text / variable	echo \$USER
clear	Clear screen	clear

B

Bash Scripting

Note

Reference: GNU Bash Manual — <https://www.gnu.org/software/bash/manual/>

Reference: ShellCheck (paste your script to find errors) — <https://www.shellcheck.net/>

What is a Bash script?

A Bash script is a plain text file containing terminal commands, run in sequence. Instead of typing the same 10 commands every time you start your robot, you put them in a script and run it with one command. Scripts power every automated workflow in robotics — launching stacks, deploying code, running health checks.

Your first script

```
hello.sh
#!/usr/bin/env bash
# hello.sh
# The first line (shebang) tells the system to run this with bash.
# Lines starting with # are comments – ignored by the shell.

echo 'Hello from ArIES!'
echo 'User:' $USER
echo 'Machine:' $(hostname)
echo 'Date:' $(date)
```

```
running the script
chmod +x hello.sh      # make it executable (only needed once)
./hello.sh             # run it

# Output:
# Hello from ArIES!
# User: ubuntu
# Machine: robot-01
# Date: Mon Jun 22 10:30:00 IST 2026
```

Variables

```
variables
#!/usr/bin/env bash

ROBOT_NAME='Aries-Bot'      # no spaces around =
MAX_SPEED=1.5

echo "Robot: $ROBOT_NAME"
echo "Max speed: ${MAX_SPEED} m/s"

# Capture command output
CURRENT_DIR=$(pwd)
echo "Directory: $CURRENT_DIR"

# Read input from the user
read -p 'Enter your name: ' NAME
echo "Hello, $NAME"
```

Tip

Always quote variables in double quotes: "\$VAR" — prevents errors when values contain spaces.

Conditionals

```
conditionals
#!/usr/bin/env bash

BATTERY=35

if [ $BATTERY -lt 20 ]; then
    echo 'CRITICAL: return to dock!'
elif [ $BATTERY -lt 40 ]; then
    echo 'WARNING: battery low'
else
    echo "Battery OK: ${BATTERY}%"
fi

# Number comparisons: -eq -ne -lt -le -gt -ge
# String comparisons: = !=
# File checks: -f (file exists) -d (dir exists)

if [ -f /opt/ros/humble/setup.bash ]; then
    echo 'ROS 2 Humble found'
fi
```

Loops

```
loops
#!/usr/bin/env bash

# for loop over a list
for SENSOR in lidar imu camera gps; do
    echo "Checking: $SENSOR"
done

# for loop with a range
for i in {1..5}; do
    echo "Run $i"
done

# loop over files
for FILE in *.py; do
    echo "Found: $FILE"
done

# while loop
COUNT=0
while [ $COUNT -lt 3 ]; do
    echo "Attempt: $COUNT"
    COUNT=$((COUNT + 1))
done
```

Functions

```
functions
#!/usr/bin/env bash

check_tool() {
    local TOOL=$1          # $1 = first argument passed to function
    if command -v $TOOL &>/dev/null; then
        echo " [OK] $TOOL"
    else
        echo " [!!] $TOOL - not found"
    fi
}

echo '=== Tool check ==='
check_tool git
check_tool ros2
check_tool python3
```

A complete example — system health check


```
robot_health.sh

#!/usr/bin/env bash
# robot_health.sh

set -e    # stop on any error

echo '=== Robot Health Check ==='
echo "Host   : $(hostname)"
echo "User    : $USER"
echo "Date    : $(date)"

DISK=$(df -h / | awk 'NR==2 {print $4}')
echo "Free disk : $DISK"

RAM=$(free -h | awk 'NR==2 {print $4}')
echo "Free RAM   : $RAM"

echo ''
echo '--- Tool check ---'
for TOOL in git python3 ros2 colcon; do
    if command -v $TOOL &>/dev/null; then
        echo "  [OK]  $TOOL"
    else
        echo "  [X]   $TOOL not installed"
    fi
done
```

C

Git — Version Control

Note

Reference: Pro Git (free book) — <https://git-scm.com/book/en/v2>

Reference: Official Git documentation — <https://git-scm.com/doc>

The problem Git solves

Without version control, a shared codebase quickly becomes `nav_v2_final_USE_THIS.py` and `nav_PRIYA_edits.py` — nobody knows which is correct and one teammate can silently overwrite another's work. Git tracks every change, who made it, and why.

Without Git	With Git
Multiple files: <code>nav_v1.py</code> , <code>nav_v2_FINAL.py</code> ...	One file with full history
No record of who changed what	<code>git blame</code> shows every edit
Overwriting teammates' work silently	Conflicts are detected and merged
'It worked last night' — no idea why it broke	<code>git log</code> + <code>git diff</code> finds the exact change

Installing and configuring Git

install and configure

```
sudo apt update && sudo apt install git -y
git --version                # confirm installation
```

One-time setup — your identity

```
git config --global user.name 'Your Name'
git config --global user.email 'you@example.com'
git config --global core.editor 'nano'
```

```
git config --list            # see current config
```

The three areas of Git

Area	What it is	Command to move there
Working Directory	Files on disk you are editing	(just edit the file)
Staging Area	Snapshot lined up for the next commit	<code>git add filename</code>
Repository (.git)	Permanent history of commits	<code>git commit -m '...'</code>

Core workflow

basic workflow

```
# 1 – create a project folder and initialise Git
mkdir my_robot_project && cd my_robot_project
git init

# 2 – create a file
echo '# My Robot Project' > README.md

# 3 – see what Git notices
git status
# Untracked files: README.md

# 4 – stage the file
git add README.md          # stage one file
git add .                  # stage everything changed

# 5 – commit (save a snapshot)
git commit -m 'chore: add README'

# 6 – see history
git log --oneline
# a1b2c3d chore: add README
```

Tip

Commit message format: type(scope): short description

feat: add LiDAR obstacle detection

fix(nav): correct odometry frame ID

chore: update .gitignore

Day-to-day commands

useful commands

```
git status          # what changed since last commit?
git diff            # exact line-by-line changes
git diff sensor.py  # changes in one file
git log            # full history
git log --oneline --graph # compact visual history

# Undo changes
git restore sensor.py # discard local edits to a file
git restore --staged file.py # un-stage a file
git reset HEAD~1      # undo last commit, keep the edits
git revert a1b2c3d     # new commit that undoes a past commit
```

.gitignore — files Git should never track

```
.gitignore
# .gitignore for a ROS 2 / ArIES project

build/
install/
log/
*.pyc
__pycache__/
*.bag
*.db3
.vscode/
*.swp
.DS_Store
```

Branches

A branch is an independent line of development. Create a branch for every new feature so that the stable code on main is never broken.

```
branches

git branch                # list branches
git checkout -b feature/lidar  # create and switch to new branch

# work, then commit
git add lidar.py
git commit -m 'feat: add LiDAR filter'

git checkout main          # switch back to main
git merge feature/lidar    # merge the feature
git branch -d feature/lidar # delete the branch
```



GitHub — Remote Collaboration

Note

Reference: GitHub Docs — <https://docs.github.com/en/get-started>

Reference: GitHub Skills (interactive) — <https://skills.github.com/>

Git vs GitHub

	Git	GitHub
What is it?	Software on your machine	Website — github.com
What does it do?	Tracks history locally	Hosts repos online, enables collaboration
Needed for other? No		No — but they work together

SSH authentication

SSH keys let you push and pull without typing a password every time.

SSH setup

```
# generate a key pair
ssh-keygen -t ed25519 -C 'you@example.com'
# press Enter for all prompts

# print your public key – copy the output
cat ~/.ssh/id_ed25519.pub

# add to GitHub:
# github.com → avatar → Settings → SSH and GPG keys → New SSH key → paste

# test the connection
ssh -T git@github.com
# expected: Hi yourname! You have successfully authenticated.
```

Connecting a local repo to GitHub

```

connect to GitHub
# on GitHub: create a new empty repository

# in your terminal
git remote add origin git@github.com:yourusername/my_robot.git
git push -u origin main
# -u sets the default – future pushes only need: git push

# the daily cycle after setup:
# 1. edit files
# 2. git add .
# 3. git commit -m 'message'
# 4. git push

```

Cloning and pulling

```

clone and pull
# clone a repo (download a full copy)
git clone git@github.com:yourusername/my_robot.git

# clone a public repo (e.g. Nav2)
git clone https://github.com/ros-planning/navigation2.git

# pull latest changes from remote
git pull

# see what remotes are set
git remote -v

```

Pull requests

A pull request (PR) is a proposal to merge your branch into main. Teammates review the code and approve before it merges. This is how every professional team and open-source ROS project works.

Step	Action
1	git checkout -b feature/my-feature
2	Make changes and commit
3	git push -u origin feature/my-feature
4	On GitHub: click Compare & pull request
5	Write a description of what you changed and why
6	Teammate reviews and clicks Merge pull request
7	git checkout main && git pull
8	git branch -d feature/my-feature

E

Advanced Git & GitHub Actions

Note

Reference: Pro Git ch. 3 (branching) and ch. 7 (tools) — <https://git-scm.com/book/en/v2>

Reference: GitHub Actions documentation — <https://docs.github.com/en/actions>

Stashing

Stash saves unfinished work temporarily so you can switch to a different task without committing incomplete code.

stashing

```
git stash                # save current changes
git stash push -m 'nav param work' # stash with a label
git stash list           # see saved stashes
git stash pop            # restore latest stash
git stash drop stash@{0} # delete a stash entry
```

Tags

tags

```
# mark a commit as a release
git tag -a v1.0.0 -m 'Competition build – Robocon 2025'

# push tags to GitHub
git push origin --tags

# list tags
git tag

# check out code at a specific tag
git checkout v1.0.0
```

Rebase

Rebase replays your commits on top of the latest main, keeping history linear. Use it on your own feature branch before opening a PR.

rebase

```
git checkout main && git pull

git checkout feature/lidar
git rebase main

# interactive rebase – clean up last 3 commits
git rebase -i HEAD~3
# in the editor: pick, squash (s), reword (r)
```

Warning

Never rebase branches that teammates are also working on (main, develop). Only rebase your own local feature branch.

GitHub Actions — auto-testing on every push

Create `.github/workflows/ci.yml` in your repo. GitHub will run this automatically on every push and pull request.

```
.github/workflows/ci.yml
# .github/workflows/ci.yml
name: CI

on: [push, pull_request]

jobs:
  build-and-test:
    runs-on: ubuntu-22.04
    steps:
      - uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.10'

      - name: Install dependencies
        run: pip install -r requirements.txt

      - name: Run tests
        run: python -m pytest tests/ -v
```

Full command reference

Command	Description
git init	Initialise a new Git repo
git status	Show changed and staged files
git add .	Stage all changes
git add file.py	Stage one file
git commit -m '...'	Commit with message
git log --oneline	Compact history
git diff	Show unstaged changes
git restore file.py	Discard local edits to a file
git reset HEAD~1	Undo last commit, keep changes
git branch	List branches

<code>git checkout -b name</code>	Create and switch to branch
<code>git checkout main</code>	Switch to main
<code>git merge name</code>	Merge branch into current branch
<code>git remote add origin URL</code>	Link local repo to GitHub
<code>git push</code>	Push commits to GitHub
<code>git push -u origin main</code>	Push and set default upstream
<code>git pull</code>	Fetch and merge from GitHub
<code>git clone URL</code>	Download a remote repo
<code>git stash</code>	Save changes temporarily
<code>git stash pop</code>	Restore stashed changes
<code>git tag -a v1.0.0 -m '...'</code>	Create an annotated tag
<code>git push origin --tags</code>	Push all tags to GitHub

F

Assignment

Complete all three tasks. Submit the link to your GitHub repository.

Task 1 — Terminal practice

Open a terminal and do the following. Include the output in your submission.

1. Print your current directory.
2. Create the folder structure `aries_devkit/scripts/` and `aries_devkit/docs/` in one command.
3. Navigate into `aries_devkit/` and create an empty file called `README.md`.
4. List all files in `aries_devkit/` with human-readable sizes.
5. Search recursively in `aries_devkit/` for any file ending in `.md`.

Task 2 — Bash scripts

Inside `aries_devkit/scripts/`, write the two scripts below. Both must be executable (`chmod +x`) and must run without errors.

setup_check.sh

```
setup_check.sh — what it must do
# 1. Print a header line
# 2. Print: hostname, current user, OS version, date
# 3. Loop over: git python3 pip3 ros2
#   Print [OK] if found, [MISSING] if not
# 4. Check free disk space – print [LOW DISK] if less than 5 GB
# 5. Print a footer line
```

git_summary.sh

```
git_summary.sh — what it must do
# 1. Accept one argument: path to a Git repo
#   Default to current directory if no argument is given
# 2. Check if the path contains a .git/ folder
#   If not, print an error and exit with code 1
# 3. Print:
#   - current branch name
#   - last 5 commit messages
#   - number of uncommitted changes
# 4. Print 'Done.'

# Useful: git branch --show-current | git log --oneline -5 | git status --short | wc -l
```

Task 3 — Git and GitHub workflow

1. Initialise a Git repo inside `aries_devkit/`.
2. Create a `.gitignore` that excludes: `*.pyc`, `__pycache__`, `*.log`, `*.bag`, `*.db3`, `build/`, `install/`.
3. Write a `README.md` with: project title, a short description, and instructions to run each script.

4. Commit everything to main with message: chore: init aries_devkit.
5. Create a public GitHub repository called aries_devkit and push your code.
6. Create a branch feature/scripts, make an improvement to one script, commit, and push.
7. Open a pull request from feature/scripts to main with a short description.
8. Merge the PR on GitHub, then run git pull locally.
9. Tag the merged commit v1.0.0 and push the tag.

Tip

Run shellcheck scripts/setup_check.sh before submitting — it catches common Bash mistakes for free.